DramAdapt QA - pre-dubbing quality gate for localized microdrama

Solution summary, 2 pages. 7 November 2026. Tharun Rathod, roll number 23115034, branch EE, IIT Roorkee - Kuku AI Generalist Assignment, Problem 3. Expanded version and every figure's source file: `docs/solution_two_pager.md`, `README.md`, `PRD.md`. Code and run artifacts: https://github.com/tharun99856/dramadapt-qa

Status, stated before the numbers. The prototype, the five checks and the validation harness are built, 109 self-tests pass, and every model run below is real with token-level logs on disk. The human half is done in part. **Step D ran and its pre-registered decision is NO-GO.** Three items are still open and are marked rather than estimated: the reviewer acceptance pass, the speaking-rate calibration, and Step E (seeded errors), which has never been run.

1. Problem and the evidence for it

AI localization can be grammatically correct and still wrong as drama: a threat becomes a request, a daughter addresses her father like a stranger, a line takes twice as long to speak as the slot allows. Those errors are cheap to fix on a script and expensive to fix after the voices exist. Kuku's own account is AI at every step of microdrama production but creative input still needing heavy human involvement (MediaNews4U), and a FICCI-EY survey of Indian media CXOs (Dec 2025) found 60% naming quality and consistency as what blocks multi-language content at scale (MediaNews4U, Mar 2026). Both are industry-wide, not microdrama-specific.

The stronger evidence is mine and it is measured. On an original 6-episode screenplay, 184 lines, a Telugu reviewer graded **81 of the 100 sampled lines** `needs_revision` in a first-pass AI localization, 19 acceptable, and rewrote 83 in their own Telugu. The 100 were drawn stratified from the 184 with minimums for short slots, address shifts, threats and split sentences, so that is the error rate inside a deliberately risk-enriched sample, not a natural rate for an unselected episode. Sheet: `data/annotation/blind_annotation.csv`.

2. What it is, and what it is not

One user (a Telugu localization editor), one step (reviewing localized dialogue before voice generation), one output (a line-by-line report with evidence and a suggested correction per flag). **Its function is quality assurance, not translation generation.** It can produce a first-pass Telugu draft when none is supplied, solely so the prototype runs without a real pipeline; that output is labelled `machine first pass - stand-in` in the UI, the report and the exported JSON. It does not dub, does not lip-sync, does not auto-approve. The human is the final gate: approve, edit or dismiss, each logged.

3. How it works

Five checks over a parsed screenplay, batched **by scene**, so request count scales with scenes, not lines.

Check	Method	Why it is separated
Meaning	LLM per scene; `same / weakened / strengthened / reversed / missing` + reason	A generic prompt averages this away; split-sentence chains are judged as whole sentences because Telugu is verb-final
Relationship & address	Python rules narrow the lines; LLM judges Telugu against a register sheet the editor confirms	Honorific systems cannot be find-and-replaced; a shift the sheet does not predict is a flag
Timing	**Python only, zero model calls.** akshara count / speaking rate vs the line's slot and the gap to the next line	Deterministic, free, reproducible; not yet calibrated
Cultural naturalness	LLM, **advisory only** - never a score or a percentage	Weakest check, so reported and priced separately
Script form	**Python only, zero model calls.** Unicode letters per line	Added after a live first pass returned 20 of 100 lines as Telugu words in Latin letters; a model sees correct Telugu there, a dubbing artist sees nothing readable

Outputs are a Markdown report and a JSON run artifact holding the inputs, every prompt, every raw response, the flags, the editor actions and the measured request and token totals. `scripts/run_qa.py --input <file or folder>` re-runs the whole gate on a screenplay nobody has seen.

4. Result, and where the failure actually sits

The alternative I said I would accept: if annotation showed the first-pass Telugu basically fine, this should shrink to the timing check alone. That premise failed - 81 of 100 lines are not fine - but the gate still returned **NO-GO**, because the pre-registered rule is that the structured arm must beat an unstructured single-prompt control at similar precision on typed flags, and it does not.

arm	flags	flag precision	reviewer issues credited (of 158)	issue lines touched (of 81)	line precision
structured, five checks	174	0.506	86	80	0.879
generic baseline control	58	0.897	109	52	0.897

No severity cut of the structured arm reaches 0.897 (high-only: 0.536). What the corrected arithmetic shows is *where* it is weak. As a line-level shortlist it is strong: it put a flag on 80 of the 81 lines the reviewer marked, one line untouched in the whole script. As a type-namer it is not: `script` found 22 of 24 with zero false positives and no model call, `timing` 18 of 19, `naturalness` 31 of 58, `relationship` 5 of 11, `meaning` 10 of 46. So the honest sentence is neither "decomposition does not work" nor "it works": this gate is good at saying **which lines to re-read** and weak at saying **what is wrong with them**, and the rule in PRD §9 measures the second thing. The decision stands as recorded and is not being relabelled. `results/scores.json`, `docs/phase0_results.md`.

5. Tools, and where they got it wrong

Python 3.13 stdlib + PyYAML, the OpenAI-compatible client against Groq's free tier (`openai/gpt-oss-120b`, temperature 0), Streamlit + pandas for the UI, `streamlit.testing.v1.AppTest` to drive it. `docs/failure_log.md` holds 26 numbered entries.

The failure that justifies the project. Every model was handed each line's slot and told to fit it. For `I set it down an hour ago.` - a 1.4 s slot - the default model returned నేను దాన్ని ఒక గంటకు ముందు పెట్టాను: **14 akshara, about 4.0 s**, and the comparison model returned 9, still over. Confident JSON, no note that the constraint was broken. Nothing about the output *looks* wrong; only measuring it shows that, which is why timing runs in plain Python and costs nothing.

The worst failure was in my own evaluation layer. While presenting the blind sheet in blocks, the assistant fabricated the content of one block - ten English lines and ten Telugu drafts that exist in no file - and the reviewer graded them in good faith; their verdicts landed on ten real rows they had never seen. It surfaced 30 rows later and a mechanical audit (does each row's real text ever appear in what was shown?) returned 90 of 100. Those ten answers were cleared, not patched, and the block was re-graded through a script that reads the CSV so row content cannot come from memory again. Three sheet-wide claims built from the ghost rows are withdrawn. The reviewer's verdicts are their own, but they were recorded against AI-drafted suggestions they reviewed and accepted, so this is human-reviewed AI-suggested annotation, not independent annotation. `docs/failure_log.md` 22, 25, 26.

A third, because it is the kind a submission hides: the assignment names `llama-3.1-8b-instant` as an example; Groq retired it from the free tier on 2026-08-16 (404 `model_not_found`). The default is a stronger model, which is the conservative direction: if the checks still find this many problems at that size, the problems are real.

6. Cost, scaling, disclosures

Measured, not estimated. 100 lines = 18 scene batches: first pass 18 requests, baseline 18, meaning/relationship/naturalness 18 each, timing and script form **zero**. Prompt tokens per request: baseline 535, meaning 1,055, naturalness 1,055, relationship 1,727, first pass 1,922 - the generic control costs ~42% of the structured checks, so any argument that structure pays has to beat that gap. The whole experiment: **134 requests, 181,911 prompt and 129,851 completion tokens, `null` dollars**, because a free tier that never billed us is not a cost. The binding constraint is the 200,000-token daily cap against ~130,000 per full pass. Two of those totals are wasted on purpose and left visible: 7 requests whose findings were thrown away, 18 re-paid because the replay guard keys on prompt text and signing off the register standard changed it (entry 26).

Scaling. The architecture is language-agnostic; the rules are not. Each new language needs its own relationship and register rules, its own segmentation logic (Telugu is verb-final), its own syllable proxy and rate calibration, and a fresh Phase 0 with a native reviewer.

Open, not finished. Single evaluator, single language, no inter-rater agreement; sample of 100; test data original rather than a production localization; the register standard is part signed off (Joss addresses Edda with నువ్వు throughout; కేన్ and హల్లోరాన్ standardised, the machine draft left untouched because saved flags quote it byte-for-byte); the speaking rate is uncalibrated (a provisional 3.5 akshara/s on the command line; `data/calibration.json` does not exist); re-priced at 5.83, a rate *derived* from the reviewer's own corrected lines against their slots, timing's 50 false positives fall to 20 and it loses 2 of its 19 true instances - so the timing column is arithmetic awaiting a stopwatch, and `scripts/derive_rate.py` says which numbers are derived and which are measured; Step E has never been run; the acceptance pass is **10 of 174 flags**, all `relationship`, and on those ten the reviewer accepted 5 where precision says 0.238 - two flags named the wrong listener, two blamed the line for what is wrong in the sheet, and four wanted a pronoun fix (మీరు where an aunt uses నువ్వు) that the check does not look at. So the rule's `checks_to_drop: ["relationship"]` now stands beside a recorded override rather than beside a blank, and the other four checks still have no acceptance number at all. Two timing statistics measured once from Kuku's confidential input pack are struck from this document and replaced by corpus numbers.